

To boldly suggest an overall plan for C++23

Abstract

Various people have lamented our lack of direction, and that we don't have a plan for the next standard (or beyond). Since I haven't heard anyone promising to propose such a plan, here goes. In a nutshell, the plan is thus: for C++23, let's work towards having the following things in that standard:

- Library support for coroutines
- Executors
- Networking

Without a particular ship vehicle yet, we should also make progress on

- Reflection
- Pattern matching

All good? Agreed? Right, carry on. :) In case you want some elaboration, read on. I will elaborate on what C++23 Must Ship and things that should progress during the standardization of C++23, but do not necessarily need to be in C++23.

C++23 must-have-do-not-fail-to-ship items

As stated in the abstract, these items are

- Library support for coroutines
- Executors
- Networking

Keen-eyed readers will notice that these are all library facilities, and the must-have list contains no language facilities. That's intentional.

Library support for coroutines

Coroutines are in C++20. The only problem is that users can't do anything with them, because there are no types that are coroutine-aware. It should be fairly non-controversial that we need to fix this problem in C++23. What exactly are the coroutine-aware types that should ship in C++23 is to-be-defined by LEWG, but in general, the expectation would be that there are task types that would be plausible, but we should also take another hard look at which existing standard types could or should be made coroutine-aware. At any rate, the standard library should provide some coroutine-supporting types out of the box.

Executors

We need executors for half-everything; we need them for networking, we probably need them for audio (if we go further in that direction), we need them for heterogeneous hardware, we need them for a better async, we need them for better task types. We have been working on executors for quite a while, and in C++23, we should deliver.

Networking

We should lower the barrier of writing networked C++ applications. We have a TS, we have years of existing practice, so once the blocker item is resolved, we should standardize networking. This shouldn't require much more rationale, but once the foundational facilities are in the standard library, writing cross-platform http code that requires no additional libraries to be built is right around the corner, and we should make that possible.

Items that should make progress, but can land later than C++23, or can land in a separate ship vehicle

Reflection

Reflection has a lot to offer: more powerful generic programming, including proxies, adapters, mediators; all sorts of things we use separate code generators for, persistence, remoting, glue code interfacing with other languages; aspect-oriented programming in general; better logging facilities, the list(s) go on and on.

Reflection (and code injection) covers a lot of ground, plugging feature holes in areas where programmers want and need to use C++, as opposed to trying to handle all this functionality with myriad smaller 'hard-coded' language extensions.

We have a TS, we have ongoing work for a constexpr-based approach, including prototype implementations. We should ship that work for the benefit of our user community, but when and in what form is not entirely clear yet.

Pattern matching

While we often say that C++ needs new control statements like it needs a hole in the head, pattern matching shows a lot of promise to deliver better filters/chains/selections than what we can approximate with pure-library facilities.

There's fair amounts of ongoing work in this area, and we should give it air time during the C++23 time frame. How to ship it, and in what form, is again not entirely clear yet.

Where are nex-gen Ranges in this plan?

We could certainly entertain more Actions and Views in the realm of Ranges. Whether such material appears for standardization is a bit unknown at this point.

What about my favorite idea X, Y, foo and bar?

We need a plan on which we focus, to make sure that the important things that we need to deliver are delivered. In order to introduce new high-priority items into a plan like this, feel free to try and convince the author of this plan that your favorite extension is

1. truly so important that it is either a must-ship item or must-progress item
2. a good idea in general.